算法设计与分析实验考试复习

理论题

1.算法的基本概念，性质及其与程序的联系与区别

算法：算法是指解决问题的一系列计算步骤，是解决方案准确而完整的描述。

算法的基本性质：

1. 输入性：有零个或多个外部量作为算法的输入。
2. 输出性：算法产生至少一个量作为输出。
3. 确定性：组成算法的每条指令必须是清晰的，无二义性的。
4. 有限性：算法中指令的执行次数有限和执行的时间有限。
5. 可行性：算法中每一条运算都必须足够基本，能够精确的执行。

程序：是算法用某种设计语言的具体实现，程序可以不满足算法的有限性。

2.算法的复杂性，如何度量，算法的渐进复杂度和阶

一个算法的评价主要从时间复杂度和空间复杂度来考虑。

时间复杂度是指执行算法所需要的时间；空间复杂度是指算法所消耗的内存空间。一般来说，计算机算法的复杂性是问题规模n 的函数f(n)，当问题规模n充分大时，我们只需考虑算法复杂性在渐近意义下的阶。

3.大O表示法的定义和渐近时间复杂度

称f(n)=O(g(n))，当且仅当存在常数c>0和n0>0，使得当n>=n0时，f(n)<=g(n)恒成立。

如果一个问题的规模是n，解决这一问题的算法所需要的时间是T(n)，它是关于n的一个函数，称其为该算法的时间复杂度，当n充分大时，T(n)的极限状态称为算法的渐近时间复杂度。足够大时.

4.几大算法的基本思想

1. 分治法：将问题规模为n的问题分解成k个规模较小的子问题，这些子问题互相独立且与原问题相同，递归地解这些子问题，然后将子问题的解合并得到原问题的解。

2. 回溯算法：在包含原问题所有解的解空间树中，用深度优先搜索的策略，从根节点出发深度搜索解空间树。当搜索到某一节点时，先判断该节点是否包含原问题的解，若包含则继续搜索下去，否则逐层向其祖先节点回溯。

3. 分支限界法：以广度优先搜索的方法搜索解空间树。每个活结点只有一次机会成为扩展节点，当成为扩展节点后一次性生成所有儿子节点，在这些儿子节点中，导致不可行解或非最优解的儿子节点被抛弃，其余的儿子节点加入到活结点表的队尾。此后从活结点表的队头取下一个节点成为扩展节点，重复上述步骤，直到找到最优解或活结点表为空时结束。

   解空间树

4. 动态规划：将原问题分解成若干个规模较小的子问题，先求解这些子问题，然后通过子问题的解得到原问题的解。

   基本步骤：

   1. 找出最优解的性质，并刻画其结构特征；
   2. 递归地定义最优值；
   3. 以自底向上的方式计算出最优值；
   4. 根据计算最优解时得到的信息，构造最优解。

   基本要素：最优子结构性质，无后效性和子问题重叠性质。

   最优子结构性质是指问题的最优解所包含的子问题的解也是最优的；

   无后效性是指某个状态以前的过程不会影响以后状态的决策，而只与当前状态有关；

   子问题重叠性质是指一个子问题在下一阶段的决策中可能会被多次使用，动态规划法正是利用了这一特性，对每一个子问题只计算一次然后将结果保存在一个表格中，当需要时只需要从表格中取出结果即可。

5. 贪心算法：贪心算法总是做出在当前看来是最好的选择，即贪心算法不是从整体最优上加以考虑，所做的选择只是在某种意义上的局部最优选择。

   基本步骤：

   1. 从问题的某个初始解出发；
   2. 采用循环语句，当可以向求解目标前进一步时，就根据局部最优策略，得到一个局部最优解，缩小问题的规模；
   3. 将所有局部最优解综合起来，得到原问题的解。

   基本要素：贪心选择性质、最优子结构性质，无后效性。

   贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择来达到。

6. 备忘录法：用表格保存已解决的子问题答案，通过自顶向下的递归方式求解，当递归到已经求解过的子问题时，直接从表格中返回子问题的解。

5.几大算法的联系与区别

1. 分治算法vs动态规划

   联系：基本思想相同，都是将原问题分解成若干个子问题，先求解子问题，然后通过子问题的解得到原问题的解。

   区别：适用于动态规划法求解的问题，经分解得到的子问题往往不是相互独立的，分治法的子问题被重复多次计算，而动态规划法将子问题的解存在一个表格中，从而避免了子问题重复计算的问题。

2. 动态规划vs备忘录法（记忆化搜索）

   联系：都是用表格保存已解决的子问题答案，避免子问题重复计算

   区别：动态规划法的递归方式是自底向上的，而备忘录法的递归方式是自顶向下的。

3. 分支限界法vs回溯法

   区别：对扩展节点采用的扩展方式不同，分支限界法采用广度优先搜索，当搜索到最优解或活结点表为空时结束，而回溯法采用深度优先搜索，只有当活结点的所有可行子节点都被遍历之后才结束。

4. 贪心算法vs动态规划

   联系：都要求问题具有最优子结构性质和无后效性

   区别：在动态规划法中，每一步所做的选择往往依赖于相关子问题的解，因此只有在求出相关子问题后才能做出选择，所采用的递归方式是自底向上的；而在贪心算法中，仅在当前状态下做出最好选择，即局部最优选择，然后再去解做出这个选择之后产生的相应子问题，所采用的递归方式是自顶向下的。

6.一些算法的一般模式

1. 回溯算法的一般模式

```
int n,a[N]; //n为考察对象的个数
void backtrack (int m)
{
    if (m>n)
        output(); //叶子节点，输出可行解
    else
       for(int i=0;i<=k;i++)     //当前节点的所有子节点
        {
            a[m]=value(i); //每个子节点的值赋值给a[m]
            if (constraint(m)&&bound(m))    //满足约束条件和限界条件
                backtrack(m+1);  //递归下一层
        }
}
```

2. 回溯法搜索子集树

```
int n,a[N]; //n为考察对象的个数
void search(int m)  //m为当前考察对象
{
    if(m>n)      //递归结束条件
        output();    //相应的输出
    else
        for(int i=0;i<=1;i++)    //控制分支数目，此处只有两个分支，0、1表示是否装入
        {
            a[m]=i;
            if(constraint(m)&&bound(m)) //剪枝函数：约束函数+限界函数 ——> 递归
                search(m+1);
        }
}
```

3. 回溯法搜索排列树

```
int n,a[N]; //n为考察对象的个数，a[N]初始化为排列的一种
void search(int m)  //m为当前考察对象
{
    if(m>n)      //递归结束条件
        output();    //相应的输出
    else
         for (int i=m;i<=n;i++)
         {
          swap(a[m],a[i]);    //调换位置
```